

JAVA PROGRAMMING

Experiment No: 01

Date: 16/09/26

Title: Library Management System Demonstrating Core Object-Oriented Programming Concepts

1. Basic Information

Experiment No	01
Date	16/09/26
Title	Library Management System Demonstrating Core Object-Oriented Programming Concepts

2. Objective / Aim

To design and implement a Java-based Library Management System that demonstrates important Object-Oriented Programming concepts including encapsulation, constructors, inheritance, method overloading, method overriding, abstraction, and interfaces.

3. System Requirements

Hardware Requirements

Component	Requirement
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Input Device	Keyboard and mouse
Display	Standard monitor / laptop display

Software Requirements

Software	Requirement
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Terminal	Command Prompt / PowerShell / Linux Terminal

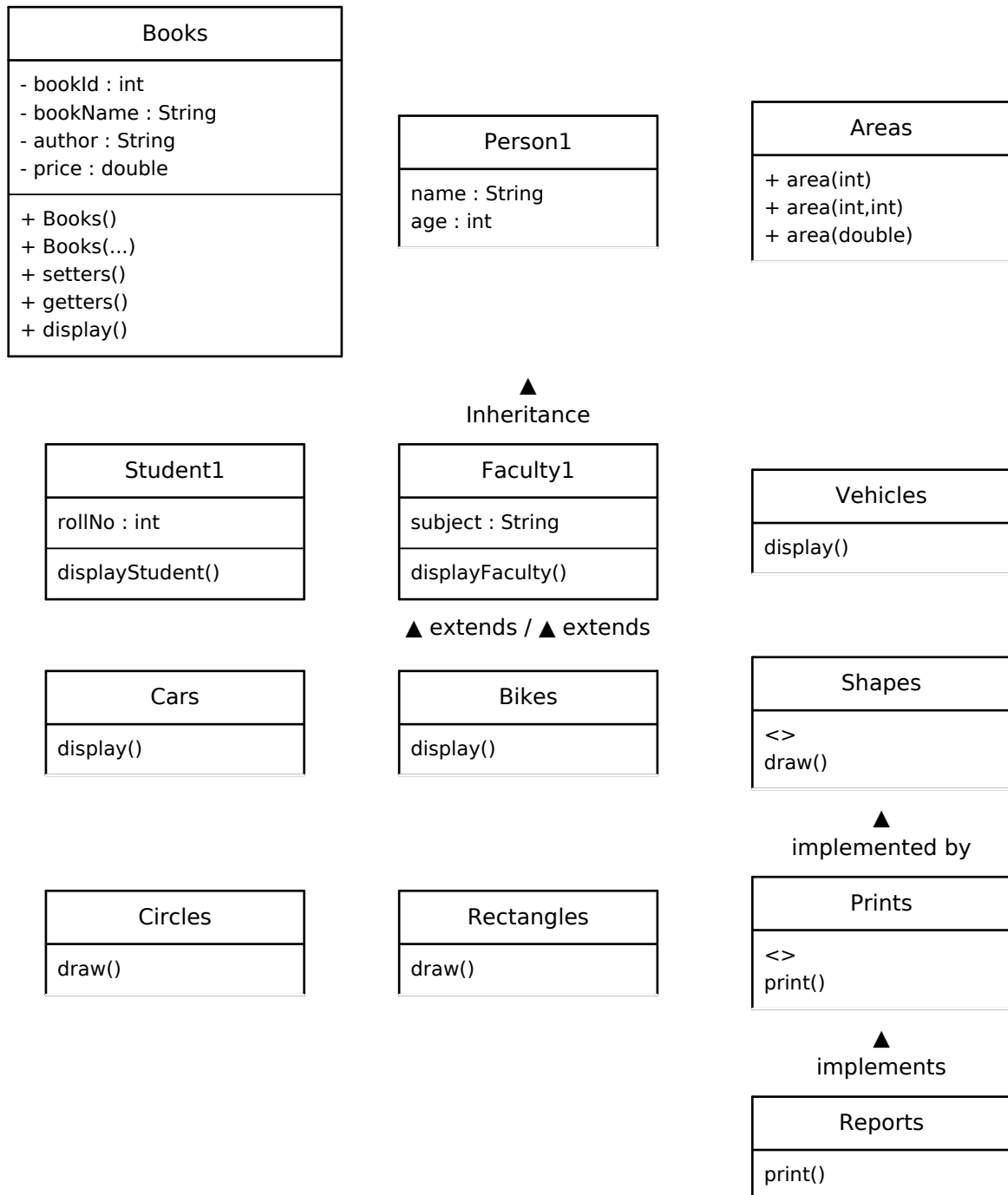
4. Algorithm / Logic Description

- Create the Books class with private fields for book ID, book name, author, and price.
- Define a default constructor and a parameterized constructor to initialize Books objects.
- Define setter methods to assign values and getter methods to access private values, demonstrating encapsulation.

4. Define Person1 as a parent class containing common name and age properties.
5. Create Student1 and Faculty1 classes that inherit from Person1, demonstrating inheritance.
6. Create the Areas class with multiple area() methods having different parameter lists, demonstrating method overloading.
7. Create the Vehicles class and override display() in Cars and Bikes, demonstrating runtime polymorphism through parent references.
8. Create the abstract Shapes class with draw(), then implement it in Circles and Rectangles.
9. Create the Prints interface with print(), and implement it in Reports.
10. In the Library main class, create objects and invoke the methods to demonstrate all the above concepts.
11. Compile the program using javac and execute it using java.
12. Verify the displayed output for constructors, book details, inheritance, overloading, overriding, abstraction, and interface implementation.

5. UML Class Diagram

The following UML-style class diagram represents the main classes, inheritance relationships, abstract class, and interface used in the program.



Note: The UML diagram shows the principal relationships used by the source program. Library is the main class containing `main()`.

6. Source Code

```
// Class Books demonstrates Encapsulation,
// Constructors, Getters and Setters
class Books {

    // Private data members
    private int bookId;
    private String bookName;
    private String author;
    private double price;

    // Default Constructor
    public Books() {
        System.out.println("Default Constructor Called");
    }

    // Parameterized Constructor
    public Books(int id, String name, String author, double price) {
        this.bookId = id;
        this.bookName = name;
        this.author = author;
        this.price = price;
    }

    // Setter methods are used to assign values
    public void setBookId(int id) {
        bookId = id;
    }

    public void setBookName(String name) {
        bookName = name;
    }

    public void setAuthor(String a) {
        author = a;
    }

    public void setPrice(double p) {
        price = p;
    }

    // Getter methods are used to access values
    public int getBookId() {
        return bookId;
    }

    public String getBookName() {
        return bookName;
    }

    public String getAuthor() {
        return author;
    }

    public double getPrice() {
        return price;
    }

    // Method to display book details
    public void display() {
        System.out.println("Book ID : " + bookId);
        System.out.println("Book Name : " + bookName);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
    }
}

// Parent class
// Demonstrates Inheritance
class Person1 {
    String name = "priya";
    int age = 22;
}

// Student1 inherits properties from Person1
class Student1 extends Person1 {

    int rollNo = 156;

    // Method to display student details
    void displayStudent() {
        System.out.println("\nStudent Details");
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Roll No : " + rollNo);
    }
}
```

```

// Faculty1 inherits properties from Person1
class Faculty1 extends Person1 {

    String subject = "Java";

    // Method to display faculty details
    void displayFaculty() {
        System.out.println("\nFaculty Details");
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Subject : " + subject);
    }
}

// Class Areas demonstrates Method Overloading
class Areas {

    // Method to calculate area of square
    void area(int side) {
        System.out.println("\nArea of Square = " + (side * side));
    }

    // Overloaded method to calculate area of rectangle
    void area(int length, int breadth) {
        System.out.println("Area of Rectangle = " + (length * breadth));
    }

    // Overloaded method to calculate area of circle
    void area(double radius) {
        System.out.println("Area of Circle = " + (3.14 * radius * radius));
    }
}

// Parent class for Method Overriding
class Vehicles {

    // Parent class display method
    void display() {
        System.out.println("\nThis is a Vehicle");
    }
}

// Cars inherits Vehicles
class Cars extends Vehicles {

    // Overriding display method
    void display() {
        System.out.println("This is a Car");
    }
}

// Bikes inherits Vehicles
class Bikes extends Vehicles {

    // Overriding display method
    void display() {
        System.out.println("This is a Bike");
    }
}

// Abstract class
abstract class Shapes {

    // Abstract method
    abstract void draw();
}

// Circles inherits abstract class Shapes
class Circles extends Shapes {

    // Implementing the abstract method
    void draw() {
        System.out.println("\nDrawing Circle");
    }
}

// Rectangles inherits abstract class Shapes
class Rectangles extends Shapes {

    // Implementing the abstract method
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}

```

```

// Interface
interface Prints {

    // Abstract method in interface
    void print();
}

// Reports implements Prints interface
class Reports implements Prints {

    // Implementing the interface method
    public void print() {
        System.out.println("\nPrinting Library Report");
    }
}

// Main class
public class Library {

    public static void main(String[] args) {

        // Creating object using Default Constructor
        Books b1 = new Books();

        // Setting book details using Setter methods
        b1.setBookId(167);
        b1.setBookName("Java Programming");
        b1.setAuthor("James Gosling");
        b1.setPrice(599);

        // Displaying first book details
        System.out.println("\nBook Details");
        b1.display();

        // Creating object using Parameterized Constructor
        Books b2 = new Books(102, "Python", "Guido", 700);

        // Displaying second book details
        System.out.println("\nParameterized Constructor");
        b2.display();

        // Creating Student object
        // Demonstrates Inheritance
        Student1 s = new Student1();
        s.displayStudent();

        // Creating Faculty object
        // Demonstrates Inheritance
        Faculty1 f = new Faculty1();
        f.displayFaculty();

        // Creating Areas object
        Areas a = new Areas();

        // Method Overloading
        // Same method name with different parameters
        a.area(6);           // Square
        a.area(10, 24);      // Rectangle
        a.area(3.7);         // Circle

        // Method Overriding
        Vehicles v;

        // Parent reference pointing to Car object
        v = new Cars();
        v.display();

        // Parent reference pointing to Bike object
        v = new Bikes();
        v.display();

        // Abstract class reference
        Shapes sh;

        // Shapes reference pointing to Circle object
        sh = new Circles();
        sh.draw();

        // Shapes reference pointing to Rectangle object
        sh = new Rectangles();
        sh.draw();

        // Creating Reports object
        // Demonstrates Interface implementation
    }
}

```

```
        Reports r = new Reports();  
        r.print();  
    }  
}
```

7. Compilation and Execution Commands

Step 1: Save the source code

Save the program as: `Library.java`

Step 2: Open terminal / command prompt

Navigate to the folder containing `Library.java`.

Step 3: Compile the program

```
javac Library.java
```

Step 4: Execute the program

```
java Library
```

Expected result: If there are no compilation errors, the program executes and displays book details followed by inheritance, overloaded area calculations, overridden methods, abstract-class output, and interface output.

8. Output

```
Default Constructor Called

Book Details
Book ID : 167
Book Name : Java Programming
Author : James Gosling
Price : 599.0

Parameterized Constructor
Book ID : 102
Book Name : Python
Author : Guido
Price : 700.0

Student Details
Name : priya
Age : 22
Roll No : 156

Faculty Details
Name : priya
Age : 22
Subject : Java

Area of Square = 36
Area of Rectangle = 240
Area of Circle = 42.99999999999999

This is a Car
This is a Bike

Drawing Circle
Drawing Rectangle

Printing Library Report
```

9. Result

The Library Management System program was implemented successfully. It demonstrates encapsulation, constructors, inheritance, method overloading, method overriding, abstraction, and interface implementation in Java.